

RAPPORT DE MISSION

Excellence Cyber

Sécurisation d'un serveur de production — Mission simulée de 7 jours

Rédigé par : AGRO Tchégoun Rachidi

Rôle simulé : Administrateur système junior, Excellence Cyber

Supervision fictive : Mme Aïssa Koudjo (RSSI) — collègue : S. Dossou

Environnement : Conteneur Docker (Ubuntu 24.04), réseau dédié 192.168.100.0/24

Serveur : srv-compta-01 (192.168.100.10)

Durée : 7 jours (25 au 31 août 2026)

Date de rédaction du rapport : 31 août 2026

Ce document raconte, jour par jour, la construction complète d'un serveur d'entreprise sécurisé — depuis la première commande jusqu'à la gestion d'un incident réel. Chaque étape technique est expliquée à deux niveaux : une explication simple, accessible à tous, puis le détail technique précis pour la traçabilité professionnelle du travail réalisé.

Note : certains identifiants et chemins sensibles ont été masqués dans cette version destinée à la diffusion publique (portfolio).

Contexte de la mission

Excellence Cyber est une entreprise fictive béninoise de conseil en cybersécurité. Dans le cadre de cette mission simulée, un nouveau serveur de production doit être sécurisé pour héberger les documents du département comptabilité (factures, données sensibles), avant l'ajout futur d'un intranet pour le département RH sur un serveur séparé. La mission impose des contraintes réalistes : accès restreint à deux administrateurs nommément identifiés, sauvegardes chiffrées avec un objectif de perte de données maximale de 24h (RPO) et un temps de restauration maximal de 30 minutes (RTO), un service métier qui ne doit jamais rester en panne, et une traçabilité complète des accès.

Pour comprendre simplement : Imagine qu'on te confie les clés d'un nouveau bureau pour une entreprise. Il faut d'abord installer une bonne serrure et ne donner les clés qu'aux bonnes personnes (Jour 1), ranger les dossiers dans des armoires bien organisées avec des étiquettes (Jour 2), faire des photocopies de sécurité de tous les documents importants et les enfermer ailleurs, au cas où un incendie détruirait le bureau (Jour 3), s'assurer que les lumières se rallument toutes seules si jamais elles s'éteignent (Jour 4), noter dans un cahier qui est entré et sorti du bureau (Jour 5), savoir réagir si quelque chose de bizarre se passe (Jour 6), et enfin faire une visite complète avant de rendre les clés au patron (Jour 7).

Jour 1 — Fondations : réseau, comptes et accès SSH sécurisé

1.1 — Le réseau dédié

Un réseau Docker isolé a été créé avec une plage d'adresses IP fixe, pour que le serveur dispose toujours de la même adresse — condition indispensable pour appliquer des règles de pare-feu stables par la suite.

```
docker network create --subnet=192.168.100.0/24 excellence_net
docker run -dit --name excellence-cyber-srv --network excellence_net \
--ip 192.168.100.10 --hostname srv-compta-01 ubuntu:24.04
```

Pour comprendre simplement : En vraie entreprise, on utilise des VLAN pour isoler les réseaux, même si les serveurs sont dans un data center à des centaines de kilomètres. Ici, on simule ce principe à plus petite échelle avec Docker : une plage d'adresses dédiée, comme un quartier réservé à l'entreprise.

1.2 — Comptes nominatifs et script d'onboarding

Trois comptes ont été créés via un script standardisé (onboard_user.sh) : deux comptes administrateurs humains (agro, dossou) avec accès sudo, et un compte de service (svc-intranet) sans shell interactif, dédié à la future application.

```
useradd -m -s /bin/bash -c "Compte admin" agro
usermod -aG sudo agro
useradd -r -s /usr/sbin/nologin -c "Compte de service" svc-intranet
```

Pour comprendre simplement : Chaque personne (ou programme) qui touche au serveur a sa propre identité, jamais un compte partagé — comme des badges d'accès personnels au lieu d'une seule clé que tout le monde se prêle. Si un jour on doit savoir qui a fait quoi, on peut le retrouver précisément.

1.3 — Authentification SSH par clés

Une paire de clés dédiée a été générée pour chaque administrateur, protégée par une passphrase. Les clés publiques ont été déployées manuellement (via docker cp) plutôt que par ssh-copy-id, car le service SSH n'était pas encore actif et l'authentification par mot de passe n'a jamais été activée — un choix de sécurité dès la conception.

```
ssh-keygen -t ed25519 -C "agro@excellence-cyber"  
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

1.4 — Durcissement SSH

```
Port 2222  
PermitRootLogin no  
PasswordAuthentication no
```

Ces trois réglages ont été testés positivement (connexion par clé réussie) ET négativement (connexion root refusée, connexion sans clé refusée) — la preuve qu'un réglage de sécurité fonctionne vraiment ne se limite jamais à un seul sens de test.

1.5 — Pare-feu UFW et segmentation réseau

Écueil rencontré et résolu : Un conteneur Docker standard ne dispose pas des permissions noyau nécessaires pour qu'UFW fonctionne (erreur « Permission denied » malgré les droits root). Solution : le conteneur a été figé dans une image (docker commit) préservant tout le travail déjà fait (comptes, clés, configuration SSH), puis recréé avec les capacités Linux NET_ADMIN et NET_RAW, qui autorisent la manipulation des règles réseau du noyau.

```
docker commit excellence-cyber-srv excellence-cyber-image:day1  
docker run -dit ... --cap-add=NET_ADMIN --cap-add=NET_RAW excellence-cyber-image:day1  
ufw allow from 192.168.100.0/24 to any port 2222 proto tcp  
ufw default deny incoming  
ufw enable
```

Résultat vérifié : seul le sous-réseau administrateur peut atteindre le port SSH, tout le reste du trafic entrant est bloqué par défaut.

Jour 2 — Stockage et séparation des données par département

Un espace de stockage dédié à la comptabilité a été créé, avec des permissions garantissant que seuls les comptes autorisés peuvent y accéder.

```
groupadd comptabilite  
mkdir -p /data/comptabilite  
chgrp comptabilite /data/comptabilite  
chmod 2770 /data/comptabilite  
usermod -aG comptabilite agro
```

Pour comprendre simplement : Le chiffre 2 devant 770 active le « bit setgid » : tout nouveau fichier créé dans ce dossier hérite automatiquement du groupe comptabilite, quel que soit qui l'a créé — comme un tampon automatique qui marque chaque nouveau document du bon service, sans qu'on ait à y penser.

L'entrée /etc/fstab correspondante a été documentée avec les options noexec et nosuid, qui empêchent l'exécution de programmes depuis ce volume — une couche de protection supplémentaire adaptée à un espace de stockage de données pures, sans raison d'y exécuter du code.

Un script de surveillance de l'espace disque (check-disk.sh) a été développé et testé dans ses deux états (normal et alerte à 80% d'utilisation). Un dossier équivalent pour le futur service RH (/data/rh) a été préparé, sans y donner accès à personne pour l'instant, en cohérence avec la décision de séparer physiquement les deux serveurs plus tard.

Jour 3 — Sauvegardes chiffrées et test de restauration

Le module le plus exigeant de la semaine. Un script de sauvegarde automatisé a été construit en trois temps : compression, chiffrement, puis suppression de la version en clair.

```
tar -czf archive.tar.gz -C /data comptabilite
gpg --batch --yes --passphrase-file /root/.[SECRET-MASQUÉ] \
--symmetric --cipher-algo AES256 -o archive.tar.gz.gpg archive.tar.gz
rm -f archive.tar.gz # jamais de version en clair conservée
find /mnt/backup -name "*.tar.gz.gpg" -mtime +7 -delete # rotation 7 jours
```

Pour comprendre simplement : On range le document dans une enveloppe (l'archive), puis on ferme cette enveloppe avec un cadenas dont seul le mot de passe secret permet l'ouverture (le chiffrement GPG). On jette ensuite l'enveloppe non fermée qui traînait — il ne doit rester que la version verrouillée.

Écueil rencontré et résolu : Comme pour UFW, systemd n'étant pas le processus principal du conteneur, les timers systemd ne fonctionnent pas. Ce choix technique a été documenté comme une décision d'architecture adaptée à un environnement conteneurisé (où cron ou un ordonnanceur externe est le standard réel de l'industrie) plutôt qu'un simple contournement — le backup a été planifié via cron.

```
@ 2 * * * /root/scripts/backup.sh
```

Le test de restauration — la preuve par les faits

Une sauvegarde qui n'a jamais été restaurée n'est qu'une hypothèse. Le dossier réel a été déplacé (simulation d'une perte totale), puis restauré depuis la dernière archive chiffrée, chronométré de bout en bout, et vérifié fichier par fichier.

```
mv /data/comptabilite /data/comptabilite_ORIGINAL_backup # simulation du désastre
LATEST=$(ls -t /mnt/backup/*.gpg | head -n 1)
gpg --batch --yes --passphrase-file /root/.[SECRET-MASQUÉ] \
--decrypt "$LATEST" > /tmp/restore.tar.gz
tar -xzf /tmp/restore.tar.gz -C /data
diff -r /data/comptabilite /data/comptabilite_ORIGINAL_backup # aucune différence
```

Résultat : restauration complète en quelques minutes, très en-dessous de l'objectif de 30 minutes (RTO), avec une intégrité des données confirmée à 100% par la commande diff.

Point d'amélioration identifié : la passphrase de chiffrement reste stockée localement sur le serveur (fichier protégé en permissions 600). En environnement de production réel, elle devrait être gérée par un coffre-fort de secrets externe (type Vault), pour ne pas être récupérable en cas de compromission totale du serveur.

Jour 4 — Résilience du service et surveillance

Un serveur web Nginx (représentant le futur intranet) a été installé, avec pour exigence qu'il redémarre automatiquement en cas de panne — une qualité appelée « résilience ».

Écueil rencontré et résolu : systemd étant absent, l'auto-redémarrage classique (Restart=on-failure) n'était pas disponible. L'outil supervisord — standard de l'industrie précisément pour ce cas de figure dans les environnements conteneurisés — a été installé et configuré à sa place.

```
[program:nginx]
command=/usr/sbin/nginx -g "daemon off;"
autostart=true
autorestart=true
startretries=3
```

Test de résilience effectué : le processus Nginx a été tué brutalement (kill -9, simulant un vrai crash). Résultat : redémarrage automatique confirmé en moins d'une seconde, avec un nouveau numéro de processus.

***Pour comprendre simplement** : C'est comme une ampoule connectée qui, dès qu'elle grille, est remplacée automatiquement et instantanément par une ampoule de secours, sans que personne n'ait besoin d'intervenir.*

Le script de surveillance du Jour 2 a été enrichi pour vérifier aussi l'état de Nginx via supervisorctl. Une distinction importante a été établie dans la planification : la sauvegarde suit une logique de RPO (une fois par jour suffit), tandis que la détection d'incident doit être rapide — vérification toutes les 5 minutes.

```
0 2 * * * /root/scripts/backup.sh # RPO quotidien
*/5 * * * * /root/check-disk.sh # détection rapide
```

Jour 5 — Traçabilité des accès

Écueil rencontré et résolu : journalctl, l'outil habituel pour consulter les journaux système, ne fonctionne pas ici (absence de journald, cohérent avec les limitations déjà rencontrées). Les commandes alternatives last, lastlog et le fichier wtmp — indépendantes de systemd — ont été utilisées avec succès.

```
last # historique des connexions : qui, quand, depuis où, durée
lastlog # dernière connexion de chaque utilisateur du système
```

Résultat : l'historique complet des connexions des administrateurs (agro, dossou) a pu être reconstitué avec précision — utilisateur, horodatage, adresse IP source — la base indispensable de toute investigation en cas d'audit ou d'incident.

***Pour comprendre simplement** : C'est le grand registre à l'entrée d'un immeuble sécurisé : chaque passage y est noté, avec le nom de la personne, l'heure d'arrivée et de départ. Si un jour on se demande qui était présent à un moment précis, il suffit de consulter ce registre.*

Jour 6 — Gestion d'un incident simulé

Un incident a été signalé : un client affirme avoir reçu une facture qu'il n'a jamais commandée. Une véritable investigation a été menée, sans connaître la réponse à l'avance.

Méthode d'investigation

- Vérification des connexions administrateur (last/lastlog) : aucune IP suspecte
- Vérification des tâches planifiées (crontab -l) : seules les tâches légitimes connues
- Vérification des journaux d'accès web Nginx : aucune requête externe suspecte
- Vérification de l'état du dossier comptable : aucune trace de génération de document

Conclusion de l'investigation : aucune preuve de compromission de l'infrastructure. Le serveur ne dispose d'ailleurs d'aucune fonctionnalité de génération ou d'envoi de factures — techniquement, l'incident ne peut pas provenir de ce serveur. L'hypothèse retenue est une tentative de phishing / usurpation d'identité ciblant directement le client, sans lien avec les systèmes d'Excellence Cyber.

***Pour comprendre simplement** : C'est comme un détective qui, plutôt que de supposer immédiatement qu'un cambriolage a eu lieu, vérifie d'abord toutes les issues, les caméras et le registre des visiteurs. Ne rien trouver d'anormal n'est pas un échec de l'enquête : c'est un résultat solide, qui oriente vers une autre explication (ici, une arnaque visant le client directement).*

Un rapport d'incident structuré (constat, investigation, preuves, conclusion, recommandations) a été rédigé et transmis, avec des conseils de prudence pour le client concerné et une recommandation d'alerte générale en interne.

Jour 7 — Audit final et bilan de mise en production

Une relecture complète de la semaine a été effectuée, façon audit de sécurité simplifié, pour vérifier qu'aucune faille évidente ne subsiste avant de considérer le serveur prêt.

Catégorie	Contrôle	Statut
Accès	Root SSH bloqué	Conforme
Accès	Authentification par clé uniquement	Conforme
Accès	Comptes nominatifs, aucun compte partagé	Conforme
Réseau	Port SSH non-standard	Conforme
Réseau	Pare-feu actif, deny par défaut	Conforme
Réseau	SSH restreint au sous-réseau admin	Conforme
Données	Séparation des périmètres (compta/RH)	Conforme
Sauvegarde	Chiffrement au repos	Conforme
Sauvegarde	Rotation automatique (7 jours)	Conforme
Sauvegarde	Restauration testée et chronométrée	Conforme
Résilience	Redémarrage automatique du service	Conforme
Traçabilité	Historique des connexions consultable	Conforme
Process	Capacité de réponse à incident démontrée	Conforme

Point d'amélioration à conserver pour une évolution future : migration de la gestion de la passphrase de sauvegarde vers un coffre-fort de secrets externe (voir Jour 3).

Bilan général de la mission

En sept jours, un serveur de production a été construit, sécurisé et validé de bout en bout, en suivant une méthodologie professionnelle complète : contrôle d'accès strict, segmentation réseau, séparation des données sensibles, sauvegardes chiffrées et testées, résilience applicative, traçabilité, et capacité de réponse à incident. Plusieurs limitations techniques propres à l'environnement conteneurisé (UFW, systemd) ont été rencontrées et surmontées par des choix d'architecture documentés et justifiés, reflétant des décisions réelles que prendrait un administrateur système face aux contraintes de son environnement.

Pour comprendre simplement : Cette mission a permis de vivre, de façon simulée mais rigoureuse, ce que fait vraiment un administrateur système ou un ingénieur sécurité au quotidien : non pas suivre une recette toute faite, mais comprendre chaque brique, la tester réellement, et savoir s'adapter quand un outil ne fonctionne pas comme prévu.